

1:1 notes



1. Introduction

The Hotel Management System API provides a comprehensive backend solution for managing hotel operations, including guest reservations, room management, staff assignments, food services, and payment processing. This RESTful API is built using Node.js, Express, and MySQL.

Key Features

Role-Based Access Control (6 user roles)

JWT Authentication

Comprehensive Room Management

Guest Reservation System

Food Ordering & Room Service

Payment Processing

Staff Task Management

Activity Logging

Prerequisites

Node.js

MySQL

Postman/Insomnia for API testing

2. Authentication

2.1 Authentication Method

The API uses JSON Web Tokens (JWT) for authentication. Include the token in the Authorization header:

text

Authorization: Bearer <your_jwt_token>

2.2 User Roles and Permissions

Role	Permissions	Access Level
Admin	Full system access	High
Super Admin		
Manager	= Managerial operations	High
receptionist	=Guest check-in/out, reservations=>>>	Medium
Housekeeping	=Room cleaning, task management	Medium
chef	= Food item management====>	Medium
waiter	= Food order management====>	Low

2.3 Environment Variables

.env

PORT=5000
DB_HOST=localhost
DB_USER=hotel_management_system
DB_PASSWORD=123123
DB_NAME=hotel_system
DB_PORT=3306
JWT_SECRET=your_super_secret_jwt_key_change_this_in_production
JWT_EXPIRES_IN=24h

3. API Endpoints

3.1 Users Management

Method	Endpoint	Description	Access	Parameters
POST	http://localhost:5000/api/users/register	Register new user	Public	

Request Body:

```
req={
  "username": "john_doe",
  "email": "john.doe@hotel.com",
  "password": "SecurePass123!",
  "first_name": "John",
  "last_name": "Doe",
  "phone": "+1234567890",
  "role": "receptionist"
} and res={
  "success": true,
  "message": "User registered successfully",
  "data": {
    "user_id": "USER_1706789123_ABC123",
    "username": "john_doe",
    "email": "john.doe@hotel.com",
```

```
    "first_name": "John",
    "last_name": "Doe",
    "role": "receptionist",
    "status": "active",
    "created_at": "2024-02-01T10:45:23Z"
  }
}
```

Post <http://localhost:5000/api/users/login> user login

req=

```
{
  "email": "netsanetabuhay@gmail.com",
  "password": "password123"
}
```

res=

```
{
  "success": true,
  "message": "Login successful",
  "token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZiZCI6IiVTU19NS1FJVVZIRlpYUUpDMCI6InJvbGUiOiJhZG1pbiIsImVtYWlsIjoibmV0c2FuZXRhYnVoYXlAZ21haWwY29tIiwiaWF0IjoxNzY5ODQxNjUxLCJleHAiOiJlE3Njk5MjgwNTF9.6qBoePnSk5gIiEUwig_m2LgzqueolGbEvQaHkQgwQzE",
  "user": {
    "user_id": "USR_MKQIUUVHFZXQJC0",
    "username": "Netsi",
    "email": "netsanetabuhay@gmail.com",
    "first_name": "Netsanet",
    "last_name": "Abuhay",
    "phone": "0922486844",
    "role": "admin",
    "status": "active",
    "created_at": "2026-01-23T06:50:50.000Z"
  }
}
```

user details

POST	http://localhost:5000/api/users/login	User login	PublicRequest
Body: email, password			

Post <http://localhost:5000/api/users> create new user

Req= {

```
{
  "username": "Gebiremariam",
  "email": "netsanet.abuhay@aastustudent.edu.et",
  "password": "password123",
  "first_name": "Sisay",
  "last_name": "Abuhay",
  "phone": "0922486844",
  "role": "admin"
}
```

```
res= {
```

```
"success": true,  
"message": "User registered successfully",  
"token":  
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6eyJlc2VyX2lkIjoiaVVNSX01MMVk2UDVXTkxLNmVZ  
HlwiidXN1cm5hbWUiOiJHZWJpcmcVtYXJpYW0iLCJlbWFPbCI6Im5ldHNhbmV0LmFidWhheUBhYXN0dXN0dWR1bn  
QuZWRR1LmV0Iiwizmlyc3RfbmFtZSI6ImlnPC2F5IiwibGFzdF9uYW11IjoiaWF0eSIsInR5cCI6IkpXVCJ9.eyJpZCI6eyJlc2VyX2lkIjoiaVVNSX01MMVk2UDVXTkxLNmVZ  
NDg2ODQ0Iiwicm9sZSI6ImFkbWluIiwic3RhZHdvZjIjoiaWN0aXZlIiwiaWF0eSIsInR5cCI6IkpXVCJ9.eyJpZCI6eyJlc2VyX2lkIjoiaVVNSX01MMVk2UDVXTkxLNmVZ  
FUMDY6NDU6MjQuMDAwWiJ9LCJyY2xiIjoiaWF0eSIsInR5cCI6IkpXVCJ9.eyJpZCI6eyJlc2VyX2lkIjoiaVVNSX01MMVk2UDVXTkxLNmVZ  
dWR1bnQuZWRR1LmV0IiwiaWF0eSIsInR5cCI6IkpXVCJ9.eyJpZCI6eyJlc2VyX2lkIjoiaVVNSX01MMVk2UDVXTkxLNmVZ  
k_GrWM7gf0mgLli27MQcoA",  
  
"user": {  
  "user_id": {  
    "user_id": "USR_ML1Y6P5WNLK5VG",  
    "username": "Gebiremariam",  
    "email": "netsanet.abuhay@aastustudent.edu.et",
```

```

        "first_name": "Sisay",
        "last_name": "Abuhay",
        "phone": "0922486844",
        "role": "admin",
        "status": "active",
        "created_at": "2026-01-31T06:45:24.000Z"
    },
    "username": "Gebiremariam",
    "email": "netsanet.abuhay@aastustudent.edu.et",
    "first_name": "Sisay",
    "last_name": "Abuhay",
    "phone": "0922486844",
    "role": "admin"
}
}

```

GET <http://localhost:5000/api/users/> Get all users Admin only

Query: page, limit, role, status

res={

```

    "success": true,
    "count": 2,
    "data": [
        {
            "user_id": "USR_ML1DDJGS4SUC1Z",
            "username": "RQE51762817",
            "email": "netsanetabuhay16@gmail.com",
            "first_name": "Netsanet",
            "last_name": "Abuhay",
            "phone": "+251922484844",
            "role": "admin",
            "status": "active",
            "created_at": "2026-01-30T21:02:52.000Z"
        },
        {
            "user_id": "USR_MKQIUUVHFZXQJC0",
            "username": "Netsi",
            "email": "netsanetabuhay@gmail.com",
            "first_name": "Netsanet",
            "last_name": "Abuhay",

```

```

        "phone": "0922486844",
        "role": "admin",
        "status": "active",
        "created_at": "2026-01-23T06:50:50.000Z"
    }
}

```

GET <http://localhost:5000/api/users/:id> Get user by ID User (own),
Admin Path: user_id

res={

```

    "success": true,
    "data": {
        "user_id": "USR_ML1Y6P5WNLK5VG",
        "username": "Dejen",
        "email": "sisayabuhay16@gmail.com",
        "first_name": "deju",
        "last_name": "abuhay",
        "phone": "0949923388",
        "role": "admin",
        "status": "active",
        "created_at": "2026-01-31T06:50:46.000Z"
    }
}

```

PUT <http://localhost:5000/api/users/:id> Update user User (own),
Admin Path: user_id, Request Body: updates

req={

```

    "first_name": "Jonathan",
    "last_name": "Doe",
    "phone": "+1234567899",
    "status": "active"
}

```

res={

```

    "success": true,

```

```

"message": "User updated successfully",
"data": {
  "user_id": "USR_ML1Y6P5WNLK5VG",
  "username": "Dejen",
  "email": "sisayabuhay16@gmail.com",
  "first_name": "deju",
  "last_name": "abuhay",
  "phone": "0949923388",
  "role": "admin",
  "status": "active",
  "created_at": "2026-01-31T06:50:46.000Z"
}
}

```

DELETE <http://localhost:5000/api/users/id> Delete user Admin only Path:
user_id

GET <http://localhost:5000/api/users/stats/> overview User statistics
Admin

Get <http://localhost:5000/api/users/search/> search user by using id,
username, first-name, last-name ,phone or any other

3.2 Guests Management

Method	Endpoint	Description	Access	Parameters
POST	http://localhost:5000/api/guests/	Create new guest	Public	Request

Body: guest details

```

req={
  "first_name": "Alice",
  "last_name": "Smith",
  "email": "alice.smith@email.com",
  "phone": "+1987654321",
  "address": "123 Main St, New York, NY 10001"
}
res={

```

```
"success": true,  
"message": "Guest created successfully",  
"data": {  
  "guest_id": "GST_1706789150_DEF456",  
  "first_name": "Alice",  
  "last_name": "Smith",  
  "email": "alice.smith@email.com",  
  "phone": "+1987654321",  
  "address": "123 Main St, New York, NY 10001",  
  "created_at": "2024-02-01T10:45:50Z"  
}  
}
```

GET <http://localhost:5000/api/guests> Get all guests PublicQuery:
page, limit, sort_by, order
res={

```
}
```

GET <http://localhost:5000/api/guests/search/:identifier> Search guests
PublicPath: identifier (ID/name/email/phone)

```
res={  
  "success": true,  
  "data": [  
    {  
      "guest_id": "GST_1706789150_DEF456",  
      "first_name": "Alice",  
      "last_name": "Smith",  
      "email": "alice.smith@email.com",  
      "phone": "+1987654321",  
      "created_at": "2024-02-01T10:45:50Z"
```

```
}  
]  
}
```

PUT <http://localhost:5000/api/guests/update/:identifier> Update guest

Admin, Manager, Receptionist Path: identifier, Request Body: updates

req={

"email": "alice.smith.new@email.com",

"phone": "+1987654333"

}

```
{  
  "success": true,  
  "message": "Guest updated successfully",  
  "data": {  
    "guest_id": "GUEST_ML1ZDMAOCJZO83",  
    "first_name": "Netsi",  
    "last_name": "ThinkPad",  
    "email": "netsi@gmail.com",  
    "phone": "0911223344",  
    "address": "ThinkPad Yoga 260, Addis",  
    "created_at": "2026-01-31T07:22:30.000Z"  
  }  
}
```

DELETE <http://localhost:5000/api/guests/delet/:id>

Delete guest

Admin, Manager Path: guest_id

res={

"success": true,

"message": "Guest deleted successfully",

"data": {

"guest_id": "GST_1706789150_DEF456",

"name": "Alice Smith"

```
}  
}
```

GET <http://localhost:5000/api/guests/stats/statistics>

Guest statistics

Admin, Manager None

```
res={  
  "success": true,  
  "message": "Guest statistics retrieved successfully",  
  "data": {  
    "total_guests": 150,  
    "daily_stats": [  
      {  
        "date": "2024-02-01",  
        "daily_count": 5,  
        "total_guests": 5  
      },  
      {  
        "date": "2024-02-02",  
        "daily_count": 8,  
        "total_guests": 13  
      },  
      {  
        "date": "2024-02-03",  
        "daily_count": 12,  
        "total_guests": 25  
      }  
    ]  
  }  
}
```

3.3 Rooms Management

Method	Endpoint	Description	Access	Parameters
GET	http://localhost:5000/api/rooms	Get all rooms	PublicQuery:	

status, room_type, min_price, max_price

```
res={
  "success": true,
  "message": "Rooms retrieved successfully",
  "data": {
    "count": 5,
    "rooms": [
      {
        "room_id": "RM_1706789200_101",
        "room_number": "101",
        "room_type": "Deluxe",
        "price": 150.00,
        "status": "available"
      }
    ]
  }
}
```

GET	http://localhost:5000/api/rooms/status/available	Get available rooms	PublicQuery: start_date, end_date, room_type
-----	---	---------------------	--

```
res={
  "success": true,
  "message": "Available rooms fetched successfully",
  "data": {
    "count": 3,
    "rooms": [
      {
```

```
    "room_id": "RM_1706789200_101",
    "room_number": "101",
    "room_type": "Deluxe",
    "price": 150.00,
    "status": "available"
  }
]
}
}
```

GET http://localhost:5000/api/rooms/search Search rooms Public

Query: q, status, min_price, max_price, amenities

```
res={
  "success": true,
  "message": "Search results retrieved successfully",
  "data": {
    "count": 1,
    "query": "101",
    "rooms": [
      {
        "room_id": "RM_1706789200_101",
        "room_number": "101",
        "room_type": "Deluxe",
        "price": 150.00,
        "status": "available"
      }
    ]
  }
}
```

GET <http://localhost:5000/api/rooms/id/:id> Get room by ID PublicPath:
room_id

GET http://localhost:5000/api/rooms/number/:room_number Get room
by number PublicPath: room_number

POST <http://localhost:5000/api/rooms> Create room Admin, Manager
Request Body: room details

```
req={
  "room_number": "102",
  "room_type": "Suite",
  "price": 250.00,
  "status": "available" // optional
}
res={
  "success": true,
  "message": "Room created successfully",
  "data": {
    "room_id": "RM_1706789300_102",
    "room_number": "102",
    "room_type": "Suite",
    "price": 250.00,
    "status": "available"
  }
}
```

PUT <http://localhost:5000/api/rooms/id> Update room Admin,
Manager Path: room_id, Request Body: updates

```
req={
  "room_number": "101A",
  "room_type": "Premium",
```

```
"price": 200.00,
"status": "maintenance"
}
res={
  "success": true,
  "message": "Room updated successfully",
  "data": {
    "room_id": "RM_1706789200_101",
    "room_number": "101A",
    "room_type": "Premium",
    "price": 200.00,
    "status": "maintenance"
  }
}
```

PATCH <http://localhost:5000/api/rooms/statusupdate/:id> Update
room status Admin, Manager, Receptionist, Housekeeping Path:
room_id, Request Body: status

```
req={
  "status": "occupied" // available, occupied, maintenance, cleaning, reserved
}
res={
  "success": true,
  "message": "Room status updated successfully",
  "data": {
    "room_id": "RM_1706789200_101",
    "room_number": "101",
    "old_status": "available",
    "new_status": "occupied"
  }
}
```

```
}
```

DELETE <http://localhost:5000/api/rooms/:id>

Delete room

Admin only Path: room_id

```
res={
  "success": true,
  "message": "Room deleted successfully",
  "data": {
    "room_id": "RM_1706789200_101",
    "room_number": "101"
  }
}
```

GET <http://localhost:5000/api/rooms/stats/>

Room statistics Admin,

Manager None

```
res={
  "success": true,
  "message": "Room statistics retrieved successfully",
  "data": {
    "totals": {
      "total_rooms": 50,
      "available_rooms": 30,
      "occupied_rooms": 15,
      "maintenance_rooms": 3,
      "cleaning_rooms": 2
    },
    "by_type": [
      {
        "room_type": "Deluxe",
        "total_rooms": 20,
        "available_rooms": 12,
```

```
"occupied_rooms": 6,  
"maintenance_rooms": 1,  
"cleaning_rooms": 1,  
"avg_price": 180.50,  
"occupancy_rate": "70.0%"  
},  
{  
  "room_type": "Suite",  
  "total_rooms": 15,  
  "available_rooms": 8,  
  "occupied_rooms": 5,  
  "maintenance_rooms": 1,  
  "cleaning_rooms": 1,  
  "avg_price": 250.75,  
  "occupancy_rate": "66.7%"  
},  
{  
  "room_type": "Standard",  
  "total_rooms": 15,  
  "available_rooms": 10,  
  "occupied_rooms": 4,  
  "maintenance_rooms": 1,  
  "cleaning_rooms": 0,  
  "avg_price": 120.25,  
  "occupancy_rate": "73.3%"  
}  
],  
"summary": {  
  "total_revenue_potential": 7850.25,  
  "average_price": 183.83,  
  "overall_occupancy_rate": "70.0%",
```

```

    "most_available_type": "Standard",
    "least_available_type": "Suite"
  }
}
}

```

3.4 Reservations Management

Method	Endpoint	Description	Access	Parameters
POST	http://localhost:5000/api/reservations	Create reservation	Admin, Manager, Receptionist	Request Body: reservation details

```

req={
  "guest_id": "GST_1706789150_DEF456", // REQUIRED
  "room_id": "RM_1706789200_101",    // REQUIRED
  "check_in": "2024-02-10",          // REQUIRED
  "check_out": "2024-02-15",         // REQUIRED
  "status": "confirmed"               // optional
}

```

```

res={
  "success": true,
  "message": "Reservation created successfully",
  "data": {
    "search_type": "reservation",
    "identifier": "RES_1706789250_GHI789",
    "count": 1,
    "reservations": [
      {
        "reservation_id": "RES_1706789250_GHI789",
        "guest_id": "GST_1706789150_DEF456",
        "room_id": "RM_1706789200_101",
        "check_in": "2024-02-10",
        "check_out": "2024-02-15",

```

```

    "status": "confirmed",
    "first_name": "Alice",
    "last_name": "Smith",
    "room_number": "101",
    "room_type": "Deluxe"
  }
]
}
}

```

GET <http://localhost:5000/api/reservations> Get all reservations
 Admin, Manager, Receptionist, Housekeeping Query: status, guest_id,
 date_from, date_to, page, limit

```

res={
  "success": true,
  "message": "Reservations retrieved successfully",
  "data": {
    "count": 5,
    "reservations": [...]
  }
}

```

GET <http://localhost:5000/api/reservations/search/:identifier> Search
 reservations Admin, Manager, Receptionist, Housekeeping Path:
 identifier (ID/guest/room/date)

```

{
  "success": true,
  "message": "Found 3 reservation(s) for guest",
  "data": {
    "search_type": "guest",

```

```
"identifier": "GST_1706789150_DEF456",
"count": 3,
"reservations": [...]
}
}
```

If search by guest-id

```
{
  "success": true,
  "message": "Reservation found successfully",
  "data": {
    "search_type": "reservation",
    "identifier": "RES_1706789250_GHI789",
    "count": 1,
    "reservations": [...]
  }
}
```

If by reservation-id

PUT <http://localhost:5000/api/reservations/:id> Update reservation

Admin, Manager, Receptionist Path: reservation_id, Request Body: updates

```
req={
  "room_id": "RM_1706789200_102",
  "check_in": "2024-02-11",
  "check_out": "2024-02-16",
  "status": "pending"
}
res={
  "success": true,
  "message": "Reservation updated successfully",
  "data": {...
```

```
...}  
}
```

PATCH <http://localhost:5000/api/reservations/:id/status/> Update
reservation status Admin, Manager, Receptionist Path: reservation_id, Request
Body: status
req={
 "status": "checked_in" // confirmed, pending, cancelled, checked_in,
 checked_out
}
res={
 "success": true,
 "message": "Reservation status updated successfully",
 "data": {
 "reservation_id": "RES_1706789250_GHI789",
 "new_status": "checked_in",
 "reservation": {...}
 }
}

DELETE <http://localhost:5000/api/reservations/:id> Delete
reservation Admin, Manager, Receptionist Path: reservation_id
res={
 "success": true,
 "message": "Reservation cancelled successfully",
 "data": {
 "reservation_id": "RES_1706789250_GHI789"
 }
}

```
}
```

GET http://localhost:5000/api/reservations/check-availability/:room_id

Check room availability PublicPath: room_id, Query: check_in, check_out

```
res={
  "success": true,
  "message": "Availability check completed",
  "data": {
    "room": {
      "room_id": "RM_1706789200_101",
      "room_number": "101",
      "room_type": "Deluxe",
      "price": 150.00,
      "status": "available"
    },
    "check_in": "2024-02-10",
    "check_out": "2024-02-15",
    "available": true,
    "message": "Room is available for selected dates"
  }
}
```

GET <http://localhost:5000/api/reservations/available-rooms> Find

available rooms PublicQuery: check_in, check_out, room_type, adults, children

```
{
  "success": true,
  "message": "Available rooms retrieved successfully",
  "data": {
```

```
"check_in": "2024-02-10",
"check_out": "2024-02-15",
"count": 3,
"rooms": [
  {
    "room_id": "RM_1706789200_101",
    "room_number": "101",
    "room_type": "Deluxe",
    "price": 150.00,
    "status": "available"
  }
]
}
```

3.5 Food Items Management

Method	Endpoint	Description	Access	Parameters
GET	http://localhost:5000/api/food-items	Get all food items	Public	

Query: category, available, vegetarian

```
res={
  "success": true,
  "message": "Food items retrieved successfully",
  "data": {
    "count": 25,
    "food_items": [
      {
        "food_id": "FD_1706789300_001",
        "name": "Grilled Salmon",
        "category": "main_course",
```

```

    "price": 25.99,
    "description": "Fresh Atlantic salmon with lemon butter sauce"
  },
  {
    "food_id": "FD_1706789310_002",
    "name": "Caesar Salad",
    "category": "salads",
    "price": 8.99,
    "description": "Romaine lettuce with Caesar dressing and croutons"
  }
]
}
}

```

GET <http://localhost:5000/api/food-items/search/:identifier> Search

food items PublicPath: identifier (ID/name/category)

```

res{
  "success": true,
  "message": "Food item found by ID",
  "data": {
    "food_id": "FD_1706789300_001",
    "name": "Grilled Salmon",
    "category": "main_course",
    "price": 25.99,
    "description": "Fresh Atlantic salmon with lemon butter sauce"
  }
}

```

If search by id

```

{
  "success": true,

```

```
"message": "Food items found in breakfast category",
"data": {
  "category": "breakfast",
  "count": 5,
  "food_items": [...]
}
}
```

If by category

```
{
  "success": true,
  "message": "Food items found by name",
  "data": {
    "search_term": "salmon",
    "count": 2,
    "food_items": [...]
  }
}
```

If by name

POST <http://localhost:5000/api/food-items> Create food item Admin,
Manager, Chef Request Body: food item details

```
req={
  "name": "Margherita Pizza",      // REQUIRED
  "category": "main_course",      // REQUIRED
  "price": 12.99,                  // REQUIRED (positive number)
  "description": "Classic pizza with tomato sauce and mozzarella" // optional
}
res={
  "success": true,
  "message": "Food item created successfully",
}
```

```
"data": {
  "food_id": "FD_1706789400_003",
  "name": "Margherita Pizza",
  "category": "main_course",
  "price": 12.99,
  "description": "Classic pizza with tomato sauce and mozzarella",
  "created_at": "2024-02-01T11:00:00Z"
}
```

PUT <http://localhost:5000/api/food-items/:id> Update food item Admin,
Manager, Chef Path: food_id, Request Body: updates

```
req={
  "name": "Premium Margherita Pizza",
  "price": 15.99,
  "description": "Updated description"
}
{
  "success": true,
  "message": "Food item updated successfully",
  "data": {
    "food_id": "FD_1706789400_003",
    "name": "Premium Margherita Pizza",
    "category": "main_course",
    "price": 15.99,
    "description": "Updated description"
  }
}
```

DELETE <http://localhost:5000/api/food-items/:id> Delete food item

Admin, Manager Path: food_id

```
res={
  "success": true,
  "message": "Food item deleted successfully",
  "data": {
    "food_id": "FD_1706789400_003",
    "name": "Premium Margherita Pizza"
  }
}
```

3.6 Food Orders Management

Method	Endpoint	Description	Access	Parameters
POST	http://localhost:5000/api/food-orders	Create food order	Public	

Request Body: order details

```
res={
  "guest_id": "GST_1706789150_DEF456", // REQUIRED
  "room_id": "RM_1706789200_101",    // REQUIRED
  "order_type": "room_service",      // optional
  "special_instructions": "No onions", // optional
  "items": [                          // REQUIRED (at least 1 item)
    {
      "food_id": "FD_1706789300_001", // Food item ID
      "quantity": 2                    // Must be > 0
    },
    {
      "food_id": "FD_1706789310_002",
      "quantity": 1
    }
  ]
}
```

```
]
}
res={
  "success": true,
  "message": "Food order created successfully",
  "data": {
    "order_id": "ORD_1706789400_001",
    "guest_id": "GST_1706789150_DEF456",
    "room_id": "RM_1706789200_101",
    "guest_first_name": "Alice",
    "guest_last_name": "Smith",
    "room_number": "101",
    "room_type": "Deluxe",
    "order_type": "room_service",
    "status": "pending",
    "total_amount": 57.97,
    "items": [
      {
        "id": 1,
        "food_id": "FD_1706789300_001",
        "food_name": "Grilled Salmon",
        "quantity": 2,
        "price": 25.99,
        "subtotal": 51.98
      },
      {
        "id": 2,
        "food_id": "FD_1706789310_002",
        "food_name": "Caesar Salad",
        "quantity": 1,
        "price": 5.99,
```

```

        "subtotal": 5.99
    }
],
    "created_at": "2024-02-01T11:00:00Z"
}
}

```

GET <http://localhost:5000/api/food-orders> Get all food orders
 Admin, Manager, Chef, Waiter, Receptionist Query: status, guest_id, date,
 order_type, page, limit

```

res={
    "success": true,
    "message": "Food orders retrieved successfully",
    "data": {
        "count": 15,
        "orders": [...]
    }
}

```

GET <http://localhost:5000/api/food-orders/search/:identifier> Search
 food orders Admin, Manager, Chef, Waiter, Receptionist Path: identifier
 (ID/guest/room)

```

{
    "success": true,
    "message": "Food order found by ID",
    "data": {...single order...}
}

```

If search by order id

```

{
    "success": true,

```

```
"message": "Food orders found for guest GST_1706789150_DEF456",
"data": {
  "guest_id": "GST_1706789150_DEF456",
  "count": 3,
  "orders": [...]
}
}
```

Response - By Name Search:

```
{
  "success": true,
  "message": "Food orders found by search",
  "data": {
    "search_term": "Alice",
    "count": 2,
    "orders": [...]
  }
}
```

If search by guest id

PUT <http://localhost:5000/api/food-orders/:id> Update food order
Admin, Manager, Chef, Waiter Path: order_id, Request Body: updates
req={
 "guest_id": "GST_1706789150_DEF456",
 "room_id": "RM_1706789200_102",
 "order_type": "dine_in",
 "status": "preparing",
 "total_amount": 65.50
}

```
res={
  "success": true,
  "message": "Food order updated successfully",
  "data": {...updated order...}
}
```

PATCH <http://localhost:5000/api/food-orders/:id/status/> Update
order status Admin, Manager, Chef, Waiter Path: order_id, Request Body:
status

```
req={
  "status": "preparing" // pending, preparing, ready, delivered, cancelled
}
res={
  "success": true,
  "message": "Food order status updated successfully",
  "data": {
    "order_id": "ORD_1706789400_001",
    "new_status": "preparing",
    "order": {...full order...}
  }
}
```

DELETE <http://localhost:5000/api/food-orders/:id> Delete food order
Admin, Manager Path: order_id

```
res={
  "success": true,
  "message": "Food order deleted successfully",
  "data": {
    "order_id": "ORD_1706789400_001",
```

```

    "guest_name": "Alice Smith",
    "room_number": "101"
  }
}

```

3.7 Payments Management

Method	Endpoint	Description	Access	Parameters
POST	http://localhost:5000/api/payments	Create payment	Admin, Manager, Receptionist	Request Body: payment details

```

req={
  "reservation_id": "RES_1706789250_GHI789", // REQUIRED: reservation_id OR
  order_id
  "order_id": null, // (Optional, can be null)
  "amount": 750.00, // REQUIRED (positive number)
  "payment_method": "credit_card", // REQUIRED
  "notes": "Paid in full with Visa", // optional
  "status": "completed" // optional (default: completed)
}

```

Request Body (one of these combinations):

```

{
  "reservation_id": null,
  "order_id": "ORD_1706789400_001",
  "amount": 57.97,
  "payment_method": "cash",
  "notes": "Room service bill",
  "status": "completed"
}

```

OR for Food Order Payment:

```
res={
  "success": true,
  "message": "Payment created successfully",
  "data": {
    "payment_id": "PAY_1706789500_001",
    "reservation_id": "RES_1706789250_GHI789",
    "order_id": null,
    "amount": 750.00,
    "payment_method": "credit_card",
    "status": "completed",
    "guest_first_name": "Alice",
    "guest_last_name": "Smith",
    "room_number": "101",
    "notes": "Paid in full with Visa",
    "created_at": "2024-02-01T11:05:00Z"
  }
}
```

GET <http://localhost:5000/api/payments> Get all payments Admin, Manager, Receptionist Query: status, guest_id, date_from, date_to, payment_method, page, limit

```
res={
  "success": true,
  "message": "Payments retrieved successfully",
  "data": {
```

```
"count": 45,
"payments": [
  {
    "payment_id": "PAY_1706789500_001",
    "reservation_id": "RES_1706789250_GHI789",
    "order_id": null,
    "amount": 750.00,
    "payment_method": "credit_card",
    "status": "completed",
    "guest_first_name": "Alice",
    "guest_last_name": "Smith",
    "room_number": "101",
    "created_at": "2024-02-01T11:05:00Z"
  }
]
}
```

GET <http://localhost:5000/api/payments/search/:identifier> Search
payments Admin, Manager, Receptionist Path: identifier (ID/guest/reference)

```
res= {
  "success": true,
  "message": "Payment found by ID",
  "data": {...single payment...}
}
```

Response - By Payment ID:

```
{
  "success": true,
```

```
"message": "Payments found by search",
"data": {
  "search_term": "Alice",
  "count": 3,
  "payments": [...]
}
}
```

Response - By Search Term:

PUT <http://localhost:5000/api/payments/:id> Update payment Admin, Manager, Receptionist Path: payment_id, Request Body: updates
req={

```
"reservation_id": "RES_1706789250_GHI789",
"order_id": null,
"amount": 800.00,
"payment_method": "bank_transfer",
"status": "refunded"
}
```

```
res={
  "success": true,
  "message": "Payment updated successfully",
  "data": {...updated payment...}
}
```

Patch <http://localhost:5000/api/payments/status/:id> Update payment status Admin, Manager, Receptionist Path: payment_id, Request Body: status

```
res={
  "status": "refunded" // pending, completed, failed, refunded
}
```

```
}
{
  "success": true,
  "message": "Payment status updated successfully",
  "data": {
    "payment_id": "PAY_1706789500_001",
    "new_status": "refunded",
    "payment": {...full payment details...}
  }
}
```

DELETE <http://localhost:5000/api/payments/:id> Delete payment

Admin only Path: payment_id

```
res={
  "success": true,
  "message": "Payment deleted successfully",
  "data": {
    "payment_id": "PAY_1706789500_001",
    "amount": 750.00,
    "payment_method": "credit_card"
  }
}
```

GET <http://localhost:5000/api/payments/stats/statistics> Payment statistics Admin, Manager None

```
res={
  "success": true,
  "message": "Payment statistics retrieved successfully",
  "data": {
    "statistics": [
```

```

{
  "date": "2024-02-01",
  "payment_method": "credit_card",
  "method_count": 15,
  "total_amount": 12500.50,
  "status": "completed",
  "status_count": 15,
  "daily_count": 20
},
{
  "date": "2024-02-01",
  "payment_method": "cash",
  "method_count": 5,
  "total_amount": 2500.00,
  "status": "completed",
  "status_count": 5,
  "daily_count": 20
}
]
}
}

```

3.8 Tasks Management

Method	Endpoint	Description	Access	Parameters
POST	http://localhost:5000/api/tasks	Create task	Admin, Manager,	
Housekeeping		Request Body: task details		

```

req={
  "title": "Clean Room 101",          // REQUIRED
  "task_type": "cleaning",            // REQUIRED
  "assigned_to": "USER_1706789123_ABC123", // optional

```

```

"room_id": "RM_1706789200_101",          // optional
"due_date": "2024-02-01",                // optional (YYYY-MM-DD)
"description": "Thorough cleaning after check-out" // optional
}
res={
  "success": true,
  "message": "Task created successfully",
  "data": {
    "task_id": "TSK_1706789600_001",
    "title": "Clean Room 101",
    "assigned_to": "USER_1706789123_ABC123",
    "assigned_to_first_name": "John",
    "assigned_to_last_name": "Doe",
    "task_type": "cleaning",
    "status": "pending",
    "room_id": "RM_1706789200_101",
    "room_number": "101",
    "room_type": "Deluxe",
    "due_date": "2024-02-01",
    "description": "Thorough cleaning after check-out",
    "created_at": "2024-02-01T11:10:00Z"
  }
}

```

GET <http://localhost:5000/api/tasks/> Get all tasks Admin, Manager, Housekeeping Query: status, assigned_to, task_type, priority, page, limit

```

res={
  "success": true,
  "message": "Tasks retrieved successfully",
  "data": {

```

```
"count": 12,
"tasks": [
  {
    "task_id": "TSK_1706789600_001",
    "title": "Clean Room 101",
    "assigned_to": "USER_1706789123_ABC123",
    "assigned_to_first_name": "John",
    "assigned_to_last_name": "Doe",
    "task_type": "cleaning",
    "status": "pending",
    "room_id": "RM_1706789200_101",
    "room_number": "101",
    "due_date": "2024-02-01",
    "created_at": "2024-02-01T11:10:00Z"
  }
]
}
```

GET <http://localhost:5000/api/tasks/search/:identifier>

Search tasks

Admin, Manager, Housekeeping Path: identifier
(ID/title/assigned_to/room)

```
req={
  "success": true,
  "message": "Task found by ID",
  "data": {...single task...}
}
```

Response - By Task ID:

```
{
  "success": true,
  "message": "Tasks found by search",
  "data": {
    "search_term": "John",
    "count": 5,
    "tasks": [...]
  }
}
```

Response - By Search Term:

PUT <http://localhost:5000/api/tasks/:id> Update task Admin, Manager,
Housekeeping Path: task_id, Request Body: updates
req={
 "title": "Deep Clean Room 101",
 "assigned_to": "USER_1706789124_DEF456",
 "task_type": "deep_cleaning",
 "status": "in_progress",
 "room_id": "RM_1706789200_101",
 "due_date": "2024-02-02",
 "description": "Updated description"
}
res={
 "success": true,
 "message": "Task updated successfully",
 "data": {...updated task...}
}

PATCH <http://localhost:5000/api/tasks/:id/status> Update task
status Admin, Manager, Housekeeping Path: task_id, Request Body:
status

req=

```
{  
  "status": "completed" // pending, in_progress, completed, cancelled  
}
```

res={

```
  "success": true,  
  "message": "Task status updated successfully",  
  "data": {  
    "task_id": "TSK_1706789600_001",  
    "new_status": "completed",  
    "task": {...full task details...}  
  }  
}
```

DELETE <http://localhost:5000/api/tasks/:id> Delete task Admin, Manager
Path: task_id

res={

```
  "success": true,  
  "message": "Task status updated successfully",  
  "data": {  
    "task_id": "TSK_1706789600_001",  
    "new_status": "completed",  
    "task": {...full task details...}  
  }  
}
```

3.9 Activities Management

Method	Endpoint	Description	Access	Parameters
GET	http://localhost:5000/api/activities	Get all activities	Admin, Manager	

Query: user_id, activity_type, date_from, date_to, page, limit

res={

 "success": true,

 "data": [

 {

 "log_id": 12345,

 "user_id": "USER_1706789123_ABC123",

 "activity": "User logged in successfully",

 "timestamp": "2024-02-01T10:45:23Z",

 "username": "john_doe",

 "first_name": "John",

 "last_name": "Doe",

 "role": "receptionist",

 "email": "john.doe@hotel.com",

 "phone": "+1234567890",

 "user_status": "active"

 }

],

 "pagination": {

 "page": 1,

 "limit": 50,

 "total": 1500,

 "totalPages": 30

 }

}

Get <http://localhost:5000/api/activities/search> search by any param

```
res={
  "success": true,
  "message": "Activities search completed successfully",
  "data": {
    "activities": [...],
    "pagination": {
      "page": 1,
      "limit": 50,
      "total": 150,
      "totalPages": 3
    }
  }
}
```

POST <http://localhost:5000/api/activities> Create activity log Admin,
Manager Request Body: activity details

```
req={
  "user_id": "USER_1706789123_ABC123",
  "activity": "Checked in guest to Room 101",
  "activity_type": "check_in",
  "module": "reservations"
}
res={
  "success": true,
  "message": "Activity logged successfully",
```

```
"data": {  
  "log_id": 12345,  
  "user_id": "USER_1706789123_ABC123",  
  "activity": "Checked in guest to Room 101",  
  "timestamp": "2024-02-01T11:00:00Z"  
}
```

Patch <http://localhost:5000/api/activities/:id> Update activity

Admin, Manager, Receptionist Path: activity_id, Request Body: updates

```
req={  
  "activity": "Updated activity description"  
}  
res={  
  "success": true,  
  "message": "Activity log updated successfully",  
  "data": {  
    "log_id": 12345,  
    "activity": "Updated activity description",  
    "timestamp": "2024-02-01T11:05:00Z"  
  }  
}
```

DELETE <http://localhost:5000/api/activities/:id> Delete activity log

Admin only Path: activity_id

```
res={  
  "success": true,  
  "message": "Activity log deleted successfully"  
}
```

GET <http://localhost:5000/api/activities/stats/overview> Activity statistics

Admin, Manager None

```
res={
  "success": true,
  "data": {
    "timeframe": "today",
    "summary": {
      "totalActivities": 150,
      "uniqueUsers": 15
    },
    "byRole": [...],
    "topUsers": [...],
    "commonActivities": [...]
  }
}
```

4. Error Codes

Status Code	Error	Description
200	OK	Request successful
201	Created	Resource created successfully
400	Bad Request	Invalid request parameters
401	Unauthorized	Authentication required or failed
403	Forbidden	Insufficient permissions
404	Not Found	Resource not found
409	Conflict	Resource already exists
422	Unprocessable Entity	Validation error
500	Internal Server Error	Server error

5. Response Formats

Success Response

json

```
{
  "success": true,
  "message": "Operation successful",
  "data": {}
}
```

Error Response

json

```
{
  "success": false,
  "message": "Error description"
}
```

Paginated Response

json

```
{
  "success": true,
  "data": [],
  "pagination": {
    "total": 100,
    "page": 1,
    "limit": 20,
    "pages": 5
  }
}
```

6. Database Schema

6.1 Tables Overview

Table Description

Primary Key

users Staff/Employee information	===>	user_id
Guests	===>Customer information	guest_id
rooms	===>Room inventory	room_id
reservations	===>Booking information	reservation_id
food_items	===>Menu items	food_id
food_orders	===>Food orders	order_id
Order_items	Order details	id
payments	===> Payment records	payment_id
tasks=>	Staff assignments	task_id
activity	===>activity_logs System activities	log_id

6.2 Field Formats

ID Generation Pattern: {PREFIX}_{TIMESTAMP}{RANDOM}

Entity Prefix Example

User USER USER_ABC123

Guest GST GST_DEF456

Room RM RM_101

Reservation RES RES_GHI789

Food Item FD FD_001

Food Order FORD FORD_JKL012

Payment PAY PAY_MNO345

Task TASK TASK_PQR678

7. Testing with Postman

7.1 Setup Environment

Create new environment "Hotel Management"

Add variables:

{base_url: <http://localhost:5000/api> }

token: (leave empty, will be set after login)

7.2 Authentication Flow

Register User: POST to <http://localhost/api/users/register/>

Login: POST to <http://localhost/api/users/login/>

Set Token: Save response token to token variable

Use **Token:** Add header: Authorization: Bearer {{token}}

7.3 Sample Test Collection Structure

text

Hotel Management System

- |— Authentication
 - | |— Register User
 - | |— Login User
- |— Users Management
- |— Guests Management
- |— Rooms Management
- |— Reservations Management
- |— Food Services
 - | |— Food Items

- | └─ Food Orders
- | └─ Payments Management
- | └─ Tasks Management
- | └─ Activities Management

8. Rate Limiting & Security

8.1 Rate Limits

Public Endpoints: 100 requests per hour per IP

Authenticated Endpoints: 1000 requests per hour per user

Admin Endpoints: 5000 requests per hour per admin

8.2 Security Headers

All responses include:

X-Content-Type-Options: nosniff

X-Frame-Options: DENY

X-XSS-Protection: 1; mode=block

Strict-Transport-Security: max-age=31536000

9. Deployment

9.1 Production Setup

Update .env file with production credentials

Set NODE_ENV=production

Use HTTPS certificates

Configure MySQL production database

Set strong JWT secret

9.2 Monitoring Endpoints

Health Check: GET <http://localhost:5000/>

Database Status: Check MySQL connection

Memory Usage: Node.js process monitoring

10. Support & Contact

For API support:

Repository: https://github.com/netsanetabuhay/Hotel_Management_System

Issues: GitHub Issues section

Author: Netsanet

Career growth



Career growth

Cadence Monthly ▾
Event 📅 Calendar event

Career goals

Goal	Action	Opportunity
Goal 1	Add action	Add opportunity
Goal 2	Add action	Add opportunity
Goal 3	Add action	Add opportunity

Achievements

Highlight	Details	Impact
Highlight 1	Add details	Add impact
Highlight 2	Add details	Add impact
Highlight 3	Add details	Add impact

Strengths

What do you do well?	Strength	Strength	Strength
Strength	Strength	Strength	Strength

Growth opportunities

What are areas of improvement?

Growth opportunity

Growth opportunity

Growth opportunity

Growth opportunity

Growth opportunity

Growth opportunity

Growth opportunity